Apuntes del Sprint 5 · Matemáticas y estadística en código: NumPy, gradiente, contrastes y CRUD

Las diapositivas del Sprint 5 mezclan matemáticas, estadística y Python en muy poco espacio. Estos apuntes **explican qué ocurre en cada paso**, con ejemplos pequeños cuyo resultado puedes comprobar a mano. Se apoyan en los apuntes del Sprint 2 (`apuntes_matematicas_y_estadistica.md`): allí está **qué es** un vector, una derivada o un intervalo; aquí **cómo se programa**. Si algo de matemáticas te suena raro, vuelve a la sección correspondiente del Sprint 2 (la tabla del final te dice cuál).

0. De qué va el sprint (en 1 minuto)

Presentación	Idea central	Cuaderno
S05_01 NumPy: calcular sin bucles	Operar con **muchos números a la vez**, sin bucles	S05_01
S05_02 Álgebra lineal con NumPy	Vectores, matrices y sistemas de ecuaciones **en código**	S05_02
S05_03 Descenso del gradiente y regresión desde cero	**Así aprende un modelo**: mide el error y da un paso cuesta abajo	S05_03
S05_04 Estadística descriptiva, Monte Carlo e intervalos de confianza	Responder al cliente **con números, no con intuiciones**	S05_04
S05_05 Contrastes de hipótesis, correlación y causalidad	Distinguir **casualidad, efecto y causa**	S05_05
S05_06 CRUD desde Python	Guardar y consultar los datos en una **base de datos**	S05_06

El reto: con la serie de ocupación de TurisData, cargar los datos en una base, mirar los datos, programar una regresión desde cero, compararla con una librería, **medir cuánta confianza merece el resultado** y responder al cliente en 150 palabras.

PARTE 1 · NumPy y álgebra lineal

1. Por qué NumPy

Los modelos de IA **no son más que muchas operaciones con vectores y matrices**. NumPy las hace **sin bucles y a gran velocidad**.

Un **array** es una **tabla de números del mismo tipo**. Tiene tres «datos de identidad»:

Atributo	Qué dice	Ejemplo
`shape`	Forma: (filas, columnas)	`(1096, 4)`
`dtype`	Tipo de los números	`float64`
`ndim`	Número de dimensiones	`2`

En el sprint se usa `ocupacion_diaria.csv`: **1.096 días** (2023-2025) con ocupación, temperatura, festivo y precio medio. Al cargarlo, `tabla.shape` da `(1096, 4)`: **filas = días, columnas = variables**.

2. Seleccionar sin bucles

Con una mini-tabla (3 filas × 2 columnas):

```
import numpy as np
tabla = np.array([[10, 20],
                  [30, 40],
                  [50, 60]])

tabla[1, 1]       # 40          → un valor (fila 1, columna 1; se empieza a contar en 0)
tabla[:, 0]       # [10 30 50]  → la columna 0 entera (":" significa "todo")
tabla[0:2, :1]    # [[10],[30]] → filas 0 y 1, solo la primera columna
```

La regla: `tabla[filas, columnas]`. `a:b` va **de a hasta b, sin incluir b**.

Máscaras: filtrar con condiciones

Una comparación devuelve **un True/False por cada elemento**. Usada como índice, **deja solo los True**:

```
ocupacion = np.array([60, 80, 70, 90, 50])
festivo   = np.array([0, 1, 0, 1, 0])
precio    = np.array([100, 130, 110, 140, 90])

ocupacion[festivo == 1].mean()                    # (80 + 90) / 2 = 85.0
ocupacion[(festivo == 0) & (precio > 95)]         # [60 70]
```

- `&` = **y**, `|` = **o**, `~` = **no**. Cada condición **entre paréntesis**.
- Segunda línea: días no festivos (60, 70, 50) **y** con precio > 95 (el de 50 tiene precio 90 → fuera) → `[60 70]`.

3. Vectorizar y broadcasting

Vectorizar = operar **con todo el array a la vez**, sin escribir un `for`.

```
ingreso_diario = ocupacion / 100 * 120 * precio     # 120 habitaciones, una cuenta por cada día
```

La primera posición sería 60/100 × 120 × 100 = 7.200 €. NumPy hace las 5 (o las 1.096) cuentas de una vez.

Estandarizar (z-score)

Pasa una variable a **«cuántas desviaciones típicas se aleja de la media»** (media 0, desviación 1):

```
ocupacion_z = (ocupacion - ocupacion.mean()) / ocupacion.std()
```

Con `[60, 80, 70, 90, 50]`: media = 70, desviación típica = 14,14. El valor 90 → (90 − 70) / 14,14 = **+1,41**; el 50 → **−1,41**; el 70 → **0**. Sirve para comparar variables con unidades distintas (ºC, %, €) y, como verás, **para que el descenso del gradiente funcione bien**.

Broadcasting: NumPy «estira» el array pequeño

Restar a una matriz `(3, 2)` un vector de 2 valores **funciona directamente**: el vector se aplica a cada fila.

```
M = np.array([[1, 2], [3, 4], [5, 6]])
M - np.array([3, 4])        # [[-2 -2], [0 0], [2 2]]
```

Así se estandariza una tabla entera: `(tabla - tabla.mean(axis=0)) / tabla.std(axis=0)`. **`axis=0` calcula por columna** (baja por las filas); **`axis=1` por fila**.

4. Aleatorios reproducibles y media móvil

```
rng = np.random.default_rng(SEMILLA)               # misma semilla → mismos números
simulada = np.clip(rng.normal(68.5, 9.6, 365), 0, 100)
```

- `rng.normal(68.5, 9.6, 365)`: 365 números «al azar» con media 68,5 y desviación 9,6.
- `np.clip(..., 0, 100)`: recorta para que no haya ocupaciones negativas ni mayores de 100.
- **Fijar la semilla** hace que **tú mañana y el cliente obtengáis el mismo resultado**: eso es **reproducibilidad**.

Media móvil: suaviza una serie promediando cada valor con sus vecinos. Con `x = [1,2,3,4,5,6]` y ventana de 3:

```
np.convolve(x, np.ones(3) / 3, mode="valid")       # [2. 3. 4. 5.]
```

Cada resultado es la media de 3 consecutivos: (1+2+3)/3 = 2; (2+3+4)/3 = 3… Con ventana 7 se ve la tendencia semanal sin el ruido diario.

5. Álgebra lineal con NumPy (repaso del Sprint 2)

Operación	En NumPy	Qué significa
Producto escalar	`a @ b`	Multiplicar elemento a elemento y **sumar**
Producto matricial	`A @ B`	(m×n) @ (n×k) = (m×k)
Transpuesta	`A.T`	Cambia filas por columnas
Inversa	`np.linalg.inv(A)`	Solo existe si el determinante no es 0

Ejemplo de producto escalar: un día con 2, 1 y 0 noches vendidas a 80, 100 y 90 €:

```
noches_dia = np.array([2, 1, 0]); precios = np.array([80, 100, 90])
noches_dia @ precios       # 2·80 + 1·100 + 0·90 = 260
```

Producto matricial: muchos productos escalares a la vez

Con los mismos precios y **cuatro días** (cada fila de la matriz es un día; cada columna, un tipo de habitación):

```
noches_dias = np.array([[2, 1, 0],
                        [1, 3, 2],
                        [0, 2, 3],
                        [3, 0, 1]])
precios = np.array([80, 100, 90])
noches_dias @ precios      # [260 560 470 330]
```

Cada fila se multiplica por el vector `precios`, igual que el producto escalar de antes: la primera fila da 2·80 + 1·100 + 0·90 = 260. El resultado es **un ingreso por día**.

La regla de las formas: `(m × n) @ (n × k) = (m × k)`; los dos `n` tienen que coincidir. Aquí `(4 × 3) @ (3,) = (4,)`: 4 días, 3 tipos de habitación, 4 ingresos. Si no coinciden, NumPy da un error `matmul: Input operand … mismatch`.

Lo mismo sirve con otro vector. Si la tasa turística por noche es 1, 1 y 2 € según el tipo, el total de los cuatro días (alojamiento + tasas) es:

```
tasa = np.array([1, 1, 2])
alojamiento = noches_dias @ precios      # [260 560 470 330]
tasas       = noches_dias @ tasa         # [3 8 8 5]
(alojamiento + tasas).sum()              # 1644
```

`.sum()` suma todos los elementos de un vector: 1.620 € de alojamiento + 24 € de tasas = **1.644 €**.

Resolver un sistema de ecuaciones

Se sabe cuánto se facturó en dos días, y cuántas noches de cada tipo (A y B) se vendieron, pero **no los precios**:

```
2·A + 1·B = 260
1·A + 3·B = 330
```

```
noches   = np.array([[2, 1], [1, 3]])
facturado = np.array([260, 330])
np.linalg.solve(noches, facturado)       # [90. 80.]  → A cuesta 90 €, B cuesta 80 €
```

Comprobación: 2·90 + 80 = 260 y 90 + 3·80 = 330 .

Inversa e identidad

La **inversa** de una matriz deshace lo que hace esa matriz. Con la tabla de noches del sistema anterior:

```
noches  = np.array([[2, 1], [1, 3]])
inversa = np.linalg.inv(noches)         # [[ 0.6 -0.2]
                                        #  [-0.2  0.4]]
np.round(noches @ inversa, 10)          # [[1. 0.]
                                        #  [0. 1.]]  ← la identidad
np.linalg.det(noches)                   # 5.0  (2·3 − 1·1)
```

Una matriz por su inversa da la **identidad**: unos en la diagonal y ceros fuera (hace el papel del 1 en una multiplicación normal). El `round` quita los restos diminutos que deja el ordenador; si ves un `-0.`, es un cero. El **determinante** (`np.linalg.det`) dice si la inversa existe: si es distinto de 0, existe. En la práctica se usa `solve` en lugar de calcular la inversa.

Cuando no hay una solución única

Imagina que otro día se vende justo el **doble** de noches de cada tipo y se factura el doble:

```
2·A + 1·B = 260
4·A + 2·B = 520
```

La segunda ecuación es la primera multiplicada por 2: **no aporta información nueva**. Hay muchos precios que cumplen las dos (A = 90 y B = 80, pero también A = 100 y B = 60). Es lo que pasa cuando una fila es múltiplo de otra:

```
M = np.array([[2, 1], [4, 2]])
det = round(np.linalg.det(M), 6)        # 0.0
tiene_inversa = det != 0                # False
np.linalg.solve(M, [260, 520])          # LinAlgError: Singular matrix
```

Determinante 0 = no hay inversa = no hay solución única. Si `solve` da «Singular matrix», mira si una fila es múltiplo de otra.

La regresión lineal es un sistema de ecuaciones (ecuación normal)

```
w = np.linalg.solve(X.T @ X, X.T @ ocupacion)
```

`X` lleva **una columna de unos** (para el término independiente) y otra con la temperatura. Ejemplo con 4 días:

Temperatura	15	20	25	30
Ocupación	90	80	70	60

Se obtiene `w = [120, −2]`: **ocupación = 120 − 2 × temperatura**. A 25 ºC: 120 − 50 = **70** (coincide con la tabla). La pendiente es **negativa** (más calor, menos ocupación).

Varias variables a la vez (mini-reto)

Añadir una variable es añadir **una columna a** `X`; la ecuación normal no cambia. Con los mismos 4 días, sabiendo ahora cuáles fueron festivo (1) y cuáles no (0):

Temperatura	15	20	25	30
Festivo	0	1	0	1
Ocupación	90	90	70	70

```
X = np.column_stack([np.ones(4), temperatura, festivo])   # unos + 2 variables
w = np.linalg.solve(X.T @ X, X.T @ ocupacion)             # [120. -2. 10.]
```

`np.column_stack` pega columnas una al lado de otra. Se obtiene `w = [120, −2, 10]`: **ocupación = 120 − 2 × temperatura + 10 × festivo**. Cada coeficiente se lee **con las demás variables sin cambiar**: a la misma temperatura, un festivo suma 10 puntos de ocupación. Comprobación con el día 2 (20 ºC, festivo): 120 − 40 + 10 = **90** (coincide con la tabla).

El mini-reto del cuaderno hace lo mismo con tres variables (temperatura, festivo y precio medio) y los 1.096 días reales: una columna más en `X`, un coeficiente más en `w`.

Para pensar: la pendiente de la temperatura era negativa (más calor, menos ocupación). ¿El calor provoca menos reservas? Lo discutiremos el próximo día.

PARTE 2 · Así aprende un modelo: descenso del gradiente

> Si no recuerdas qué es una derivada o un gradiente, relee las secciones de derivadas y gradiente de los apuntes del Sprint 2. Aquí solo lo usamos.

6. La derivada: la pendiente en un punto

- **Positiva:** la curva sube. **Negativa:** baja. **Cero:** estás en un mínimo o un máximo.

Se puede **aproximar sin fórmulas** con un paso `h` muy pequeño:

```
def derivada_numerica(func, x, h=1e-6):
    return (func(x + h) - func(x - h)) / (2 * h)
```

Es literalmente «la pendiente de un trocito minúsculo de curva». Para `g(x) = x² − 6x + 20`, en `x = 10` da ≈ 14 (la fórmula exacta: 2·10 − 6 = 14).

7. La función de coste: el error cuadrático medio (MSE)

El modelo necesita **un número que diga cómo de mal lo está haciendo**:

```
def mse(y_real, y_pred):
    return np.mean((y_real - y_pred) ** 2)
```

Ejemplo: reales `[2, 4, 6]`, predicciones `[3, 4, 5]`. Errores: −1, 0, +1 → al cuadrado: 1, 0, 1 → media = **2/3 = 0,67**.

¿Por qué al cuadrado? Para que **los errores grandes pesen más** y **no se cancelen** (un +1 y un −1 sumarían 0 y parecería perfecto).

El modelo ingenuo (referencia a batir): predecir siempre la media. Con `[2, 4, 6]` (media 4) el MSE es (4 + 0 + 4)/3 = **2,67**. Tu modelo debe mejorar esa cifra; si no, no aporta nada.

8. Descenso del gradiente en una dimensión

La idea: estás en una montaña con niebla y quieres bajar al valle. No ves el valle, pero **notas hacia dónde baja el suelo bajo tus pies**. Das un paso en esa dirección y repites.

```
x ← x − tasa · pendiente
```

Ejemplo: `g(x) = x² − 6x + 20` (mínimo en x = 3; derivada = 2x − 6). Empezamos en x = 10 con tasa 0,1:

Paso	x	Pendiente 2x − 6	Nuevo x = x − 0,1·pendiente
1	10	14	10 − 1,4 = **8,6**
2	8,6	11,2	8,6 − 1,12 = **7,48**
3	7,48	8,96	7,48 − 0,896 = **6,584**
…	…	…	…
50			**≈ 3,0001**

Cada paso se acerca al mínimo (3) y los pasos se hacen más pequeños porque la pendiente baja.

La tasa de aprendizaje: el tamaño del paso

Tasa	Qué pasa
Muy pequeña	Llega, pero **lentísimo**
Buena	Converge en pocos pasos
Muy grande	**Se pasa del mínimo y puede divergir**

Con tasa = 1,1 desde x = 10: tras 5 pasos x ≈ **−14,4**, y cada paso se aleja más. **Diverge.**

9. Regresión lineal con gradiente

Ahora el «terreno» es el error MSE y lo que se ajusta son **los pesos** `w` del modelo.

- **Modelo:** `ocupación ≈ w0 + w1 · temperatura`, es decir `y_pred = X @ w`.
- **Gradiente del MSE:** `(2 / n) · Xᵀ (X w − y)`.
- **Estandarizamos** la temperatura (media 0, desviación 1) para que el descenso sea **estable**: si una variable va de 0 a 1.000 y otra de 0 a 1, el «valle» queda estirado y cuesta bajar.

```
def gradiente_mse(X, y, w):
    return (2 / len(y)) * X.T @ (X @ w - y)

def entrenar(X, y, tasa=0.1, epocas=200):
    w = np.zeros(X.shape[1])            # empieza con pesos cero
    historial = []
    for _ in range(epocas):
        w = w - tasa * gradiente_mse(X, y, w)        # un paso cuesta abajo
        historial.append(mse(y, X @ w))              # apunta el error
    return w, historial
```

Cada **época** es una vuelta completa: calcular el gradiente → dar un paso → anotar el error. **La curva `historial` debe bajar.**

Ejemplo mínimo: con los puntos `(1,2), (2,4), (3,6)` (la recta `y = 2x`), el entrenamiento converge a `w ≈ [0, 2]`: intercepto 0 y pendiente 2.

Comprobación de seguridad: el `assert` compara **tu gradiente con el calculado por derivadas numéricas**. Si no coinciden, hay un error en la fórmula.

Elegir la tasa: qué mirar si algo va mal

Síntoma	Causa	Arreglo
El error **sube o sale** `nan`	Tasa demasiado grande	**Dividirla entre 10**
Baja, pero **muy despacio**	Tasa pequeña o variables sin estandarizar	Estandarizar; más épocas
Pendientes absurdas en unidades originales	Olvidaste deshacer la estandarización	`coef / desviación`

Deshacer la estandarización y comparar con una librería

Si entrenaste con la temperatura estandarizada, para volver a «puntos por grado»:

```
pendiente  = w[1] / desv
intercepto = w[0] - w[1] * media / desv
np.isclose(pendiente, lr.coef_[0], atol=1e-3)      # ¿coincide con scikit-learn?
```

Si tu modelo «desde cero» **coincide con** `LinearRegression`, **has entendido cómo aprende un modelo**. Todo lo que viene después (redes neuronales incluidas) es **este mismo bucle con más pesos**.

PARTE 3 · Estadística con Python y CRUD

10. Describir los datos

Tipo de medida	Qué mide	Cómo
Centro	Dónde está el «medio»	Media (sensible a extremos) y **mediana** (el valor central)
Dispersión	Cuánto se separan los datos	Desviación típica; rango intercuartílico (IQR = percentil 75 − percentil 25)
Forma	Si es simétrica	Asimetría (*skew*); `describe()` da un resumen rápido

Ejemplo de por qué importa la mediana: noches de 5 reservas: `[2, 3, 4, 5, 100]` (un grupo grande). La **media** = 22,8 (engañosa); la **mediana** = 4 (representa a la mayoría).

```
resumen = {"media": serie.mean(), "mediana": serie.median()}
media_por_mes = df.groupby("mes")["ocupacion_pct"].mean()     # una media por cada mes
```

11. Distribuciones y simulación Monte Carlo

Con `scipy.stats` se calculan probabilidades de distribuciones conocidas. `norm` (normal) y `binom` (binomial) tienen:

Función	Responde a
`.cdf(x)`	P(valor ≤ **x**)
`.sf(x)`	P(valor > **x**) (lo contrario de cdf)
`.ppf(p)`	El valor `x` que deja una probabilidad `p` por debajo

Ejemplo con TurisData: la tasa de cancelación es 45,9 %. Si hay 20 reservas, ¿cuál es la probabilidad de que se cancelen 10 **o más**?

```
stats.binom.sf(9, 20, 0.459)       # ≈ 0.44  → un 44 %
```

(`sf(9)` = P(más de 9) = P(10 o más).)

Monte Carlo: «si la fórmula es difícil, simula muchas veces y cuenta»

Pregunta del cliente: *vendemos 170 reservas para 100 habitaciones; ¿qué probabilidad hay de tener que reubicar a alguien (que se presenten más de 100)?*

Cada reserva se presenta con probabilidad 1 − 0,459 = 0,541. Se simulan **10.000 días**:

```
presentados   = (rng.random((10000, 170)) < (1 - 0.459)).sum(axis=1)
prob_reubicar = (presentados > 100).mean()          # ≈ 0.09
```

Se leen así: la comparación genera True/False por reserva y día; `sum(axis=1)` cuenta cuántas se presentan cada día; `(presentados > 100).mean()` es la **proporción de días con más de 100**. En media se presentan 170 × 0,541 ≈ **92** personas, con desviación ≈ 6,5, y superar 100 ocurre en **≈ 9 % de los días**.

> **La estadística da el número (≈ 9 %); la decisión es del cliente.** ¿Compensa el riesgo de reubicar al 9 % de los días por llenar más habitaciones? Eso lo decide el negocio, no el código.

12. Intervalos de confianza

Un **intervalo de confianza (IC) al 95 %** es **un rango de valores plausibles** para la media (o la proporción) verdadera.

> **NO** significa «hay un 95 % de probabilidad de que el valor verdadero esté aquí». Significa que, **si repitiéramos el estudio muchas veces, el 95 % de los intervalos construidos así contendrían el valor verdadero**.

Ejemplo (proporción): de 6.000 reservas, $\hat{p}$ = 0,459 canceladas.

```
margen = 1.96 * np.sqrt(p_hat * (1 - p_hat) / n)      # 1,96 · √(0,459·0,541/6000) ≈ 0,0126
ic = (p_hat - margen, p_hat + margen)                  # ≈ (0,446 ; 0,472)
```

Se puede decir: «la tasa verdadera está **entre el 44,6 % y el 47,2 %**».

El tamaño de la muestra manda: con solo 100 reservas y $\hat{p}$ = 0,4, el margen sería 1,96·√(0,4·0,6/100) ≈ **0,096**: el IC iría de **30 % a 50 %** (mucho más ancho). Más datos → intervalo más estrecho → más seguridad.

Para una media (por ejemplo, la ocupación de un mes) se usa:

```
stats.t.interval(0.95, len(x) - 1, loc=x.mean(), scale=stats.sem(x))
```

Con 8 días de ocupaciones `[70,72,68,75,65,71,69,74]`: media 70,5 y IC al 95 % ≈ **(67,8 ; 73,2)**.

13. Contrastes de hipótesis

La lógica es un **juicio con presunción de inocencia**:

1. **H0 (hipótesis nula):** «no hay diferencia» (la ocupación es igual en festivos y en días normales).
2. Calculas el **p-valor**: *la probabilidad de ver una diferencia así de grande si H0 fuera cierta***.
3. Si **p < 0,05**, **rechazamos H0**: la diferencia es poco probable que sea casualidad.

```
stats.ttest_ind(festivos, normales, equal_var=False)      # compara dos medias
stats.chi2_contingency(tabla_canal)                       # ¿el canal influye en cancelar?
```

- **t de Student:** compara **medias** de dos grupos.
- **Chi-cuadrado:** comprueba si **dos categorías están relacionadas** (canal y cancelación).

Significativo no es lo mismo que importante

- El **p-valor** dice si es **casualidad o no**.
- El **tamaño del efecto** dice **cuánto importa**. d de Cohen = (media A − media B) / desviación combinada. Referencia: **0,2 pequeño · 0,5 medio · 0,8 grande**.

Ejemplo: con 6.000 reservas, una diferencia minúscula puede dar p = 0,001 (muy «significativa») y no tener ningún valor práctico. Con pocos datos, una diferencia enorme puede no salir significativa. **Mira siempre las dos cosas.**

> **Ojo con el p-valor:** *no* es la probabilidad de que H0 sea cierta. Y «no rechazo H0» **no** demuestra que no haya diferencia: quizá faltaban datos.

14. Correlación no es causalidad

Pregunta	Respuesta
Correlación calor ↔ ocupación	**−0,63** (más calor, menos ocupación)
¿Hay que «enfriar» el hotel para vender más?	**No**: el **invierno es temporada alta** y es la época más fresca

El **mes** influye a la vez en la temperatura y en la ocupación: es una **variable de confusión**. Al **controlar el mes** en una regresión múltiple:

```
smf.ols("ocupacion_pct ~ temperatura_c + festivo + C(mes)", data=df).fit()
```

el efecto de la temperatura **se diluye**, mientras que el del **festivo (≈ +9 puntos) resiste**.

> Con datos observacionales **no se demuestra causalidad**: se **descartan explicaciones simples**. Para demostrar causa hace falta un experimento.

15. Un vistazo bayesiano

En vez de un p-valor: **creencia inicial + datos = creencia actualizada**. Para una tasa con `c` cancelaciones de `n` reservas, la creencia actualizada es una **Beta(1 + c, 1 + n − c)**.

```
post_web      = rng.beta(1 + c_web, 1 + n_web - c_web, 10000)
prob_web_mayor = (post_web > post_directo).mean()
```

Ejemplo pequeño: web con 3 canceladas de 10, directo con 1 de 10. Simulando se obtiene `prob_web_mayor ≈ 0,84`: «*hay un 84 % de probabilidad de que la web cancele más que el directo*». **Es una frase que entiende un cliente**, a diferencia del p-valor.

16. CRUD desde Python (el paso 1 del reto)

Es lo mismo que el CRUD en SQL del Sprint 3, pero **escrito desde Python**:

```
import sqlite3
con = sqlite3.connect("turisdata.db")
con.execute("SELECT id_reserva FROM reservas WHERE id_cliente = ?", (id_cliente,))
con.execute("UPDATE reservas SET cancelada = 1 WHERE id_reserva = ?", (5046,))
con.commit()                 # confirmar; con.rollback() lo deshace
diaria.to_sql("ocupacion_diaria", con, if_exists="replace", index=False)    # CSV → base
```

- **Nunca pegues texto directamente en el SQL** (`"... WHERE id = " + dato`): usa `?` y pasa los valores aparte. Así evitas la **inyección SQL** (que alguien escriba código malicioso en un campo).
- **Nada es definitivo hasta** `commit`; `rollback` lo deshace; `rowcount` dice **cuántas filas** tocaste (si esperabas 1 y salen 6.000, algo va mal).
- El mismo CRUD se hace en **MongoDB** con `pymongo` / `mongomock`.

---

17. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Repaso de teoría	Practica en
Arrays, máscaras, broadcasting	1-4	—	`S05_01_numpy`
Producto matricial, sistemas, ecuación normal	5	Sprint 2: vectores y matrices	`S05_02_algebra_lineal`
Derivada, gradiente, tasa de aprendizaje	6-9	Sprint 2: derivadas y gradiente	`S05_03_gradiente_regresion`
Media, mediana, IC, Monte Carlo	10-12	Sprint 2: estadística	`S05_04_estadistica_descriptiva_ic`
p-valor, efecto, confusión, Bayes	13-15	Sprint 2: Bayes, sesgos y Simpson	`S05_05_contrastes_causalidad`
CRUD en Python	16	Sprint 3: SQL y NoSQL	`S05_06_crud_python`
Todo junto	todas	—	Reto `S05_07_reto_ocupacion`

18. Chuleta de vocabulario

Palabra	En lengua normal
Array	Tabla de números del mismo tipo
shape	Forma del array (filas, columnas)
Vectorizar	Operar con todo el array a la vez, sin bucles
Broadcasting	NumPy «estira» el array pequeño para poder operar
Máscara	Filtro de True/False para elegir elementos
Semilla	Número que hace que el azar sea repetible
z-score	Cuántas desviaciones típicas se aleja un valor de la media
Derivada	Pendiente en un punto
Gradiente	Derivadas de varias variables: la dirección de máxima subida

Palabra	En lengua normal
MSE	Error cuadrático medio
Tasa de aprendizaje	Tamaño del paso en el descenso del gradiente
Época	Una vuelta completa de entrenamiento
Monte Carlo	Simular muchas veces y contar
IC (intervalo de confianza)	Rango de valores plausibles para el valor verdadero
H0	Hipótesis nula: «no hay diferencia»
p-valor	Probabilidad de ver esto si H0 fuera cierta
Tamaño del efecto	Cuánto importa una diferencia
Variable de confusión	Algo que influye en las dos variables y crea una falsa relación
Inyección SQL	Atacar una base metiendo código en un campo de texto
commit / rollback	Confirmar / deshacer cambios en la base

19. Cómo estudiar estos apuntes

1. **Antes de cada cuaderno**, lee la sección que te indica la tabla del apartado 17.
2. **Comprueba a mano los ejemplos pequeños** (media 70, z = 1,41; los pasos del gradiente): si los entiendes con 3 números, los entenderás con 1.096.
3. **Ejecuta y cambia valores** (la tasa, la semilla) para ver qué se rompe.
4. **En el reto, no basta con el resultado:** hay que decir **cuánta confianza merece** (intervalo, tamaño de efecto) y **qué no se puede concluir** (causalidad).